

中国研究生创新实践系列大赛

“华为杯”第二十三届中国研究生

数学建模竞赛

题 目： 多核神经网络处理器的计算图切分与调度优化

---

摘 要：

面向多核神经网络处理器的计算图调度需求，本文研究子图划分、核心分配与执行顺序的协同优化。评测以最小化仿真完成时间（Makespan）为首要目标，兼顾外部主存（DDR）数据搬运量，并将端到端求解耗时作为算法效率指标。赛题三问对应不同的芯片存储与执行规则；本文依据图拓扑结构构造少量候选方案，在适用条件成立的分支引入理论下界辅助剪枝，并由模拟器在受限预算内比较完整执行时间择优。算法在全部 100 张基准图与 1 至 5 核心（三题共 1500 种测试组合）上均生成了合法方案。

**针对问题一：**在子图独立封装为任务、跨任务数据必须经外部主存中转的场景下，本文通过连通分量与拓扑前沿生成少量结构化候选并实施多级细化；通过桥树分解识别可迁移的独立计算分支在核心间重分配；基于拓扑高度波次严格保证执行时序无环，消除跨核循环等待。5 核心场景下，100 张基准图相对官方单核基准的逐用例平均加速比达到 4.0553。

**针对问题二：**在同一核心内子图合并为单一任务、允许私有缓存跨子图复用数据的场景下，本文协同优化计算分配与数据流转。针对片上缓存容量受限问题，利用流水线空闲区间紧凑编排计算；建立超图连接度模型作为划分引起的数据跨核心复制量代理（区别于最终额外 DDR 搬运量）；在二核心局部调整中设计受流水线工作量上限保护的最小割方法，防止算力向单核倾斜。5 核心场景下，100 张基准图相对官方单核基准的逐用例平均加速比达到 4.5498。

**针对问题三：**在同一核心内子图合并并引入共享只读缓存的场景下，本文协同优化算子顺序与缓存利用。针对多分支树状归约结构，在子树不交错且内部开销固定的受限模型下，证明按子树内部峰值与保留输出之差（ $h_i - r_i$ ）降序排列可使模型内临时结果存储峰值达到最小；构建基于硬件流水线必要不等式的时间下界快速剪枝，在至多 3 次在线评测预算内按 Makespan 严格改善选优。5 核心场景下，100 张基准图相对官方单核基准的逐用例平均加速比达到 4.7576；针对同一方案开展同配置成对仿真，开启只读缓存相比关闭缓存的逐用例成对平均加速比达到 1.0829，平均字节命中率为 31.36%。

在共享多核主机环境下，三道题目各 500 种测试组合的端到端平均求解墙钟时间分别为 P1 2.825 秒、P2 4.029 秒与 P3 4.401 秒。测试所报告的方案完成时间为离散事件仿真时钟周期，求解耗时为通用计算机上的墙钟时间，数据反映基准仿真推进结果，而非物理芯片板级实测。

**关键词：** 计算图切分；多核调度；微架构仿真；超图最小割；树状存储排序；理论下界

# 目录

|          |                         |           |
|----------|-------------------------|-----------|
| <b>1</b> | <b>问题重述与分析</b>          | <b>5</b>  |
| 1.1      | 问题背景与优化目标               | 5         |
| 1.2      | 计算图与硬件执行约束              | 5         |
| 1.2.1    | 计算图表示与数据依赖              | 5         |
| 1.2.2    | 执行资源与计算通信重叠             | 6         |
| 1.2.3    | 子图划分与调度方案               | 7         |
| 1.3      | 问题分析与求解思路               | 8         |
| <b>2</b> | <b>模型假设与符号说明</b>        | <b>9</b>  |
| 2.1      | 模型假设与执行约束               | 9         |
| 2.2      | 张量与物理副本的表示              | 10        |
| 2.3      | 符号说明                    | 10        |
| 2.4      | 评价指标与计算方法               | 12        |
| 2.4.1    | 完成时间与多核加速比              | 12        |
| 2.4.2    | 额外 DDR 搬运量与缓存命中率        | 13        |
| 2.4.3    | 端到端求解耗时                 | 13        |
| 2.4.4    | 评测方法与调用预算               | 13        |
| <b>3</b> | <b>多核调度的数学模型与求解框架</b>   | <b>14</b> |
| 3.1      | 多核切分与调度模型               | 14        |
| 3.1.1    | 决策变量与方案表示               | 14        |
| 3.1.2    | 数据依赖与执行顺序约束             | 14        |
| 3.1.3    | 目标函数与可行域                | 15        |
| 3.2      | 求解框架与方法选择               | 15        |
| 3.2.1    | 结构化候选方案的构造与筛选           | 15        |
| 3.2.2    | 相关研究与方法适用性              | 16        |
| 3.3      | 可行性判定与完成时间下界            | 18        |
| 3.3.1    | 扩展依赖图的无环性判定             | 18        |
| 3.3.2    | 完成时间下界与候选剪枝             | 18        |
| 3.4      | 求解耗时与计算复杂度              | 20        |
| <b>4</b> | <b>问题一：独立任务执行下的多核调度</b> | <b>21</b> |
| 4.1      | 问题分析与时序约束               | 21        |
| 4.2      | 结构化候选方案的构造              | 22        |
| 4.2.1    | 基础候选方案                  | 22        |
| 4.2.2    | 多级细化与在线择优               | 24        |
| 4.3      | 基于支路重分配的调度优化            | 25        |
| 4.3.1    | 拓扑波次与支路识别               | 26        |
| 4.3.2    | 执行全序的构造                 | 28        |
| 4.3.3    | 调度依赖图的无环性证明             | 29        |
| 4.4      | 候选规模与采纳准则               | 30        |
| 4.5      | 结果与分析                   | 30        |

|          |                         |           |
|----------|-------------------------|-----------|
| 4.5.1    | 全量用例的性能比较               | 30        |
| 4.5.2    | 典型算例的执行时序分析             | 31        |
| <b>5</b> | <b>问题二：数据复用下的多核调度</b>   | <b>34</b> |
| 5.1      | 问题分析与数据复用约束             | 34        |
| 5.2      | 基于超图的复制量模型              | 37        |
| 5.2.1    | 超图表示与适用条件               | 37        |
| 5.2.2    | 超边与复制代价                 | 37        |
| 5.2.3    | 链分组移动的代价增量              | 39        |
| 5.3      | 基于局部最小割的模型求解            | 39        |
| 5.3.1    | 基于空闲区间的初始分配             | 39        |
| 5.3.2    | 局部超图割与辅助流网络             | 39        |
| 5.3.3    | 流水线工作量约束与回退             | 42        |
| 5.3.4    | 候选方案生成与择优               | 43        |
| 5.3.5    | 时间复杂度分析                 | 46        |
| 5.4      | 结果与分析                   | 46        |
| 5.4.1    | 全量用例的性能比较               | 46        |
| 5.4.2    | 搬运量与完成时间的权衡分析           | 48        |
| <b>6</b> | <b>问题三：共享只读缓存下的多核调度</b> | <b>53</b> |
| 6.1      | 问题分析与缓存访问约束             | 53        |
| 6.2      | 基于归约树的存储峰值模型            | 54        |
| 6.2.1    | 归约树表示与存储峰值              | 54        |
| 6.2.2    | 非交错子树的最优排序及证明           | 59        |
| 6.3      | 基于归约树排序的模型求解            | 61        |
| 6.3.1    | 归约树识别与执行序列构造            | 61        |
| 6.3.2    | 分层候选方案构造                | 62        |
| 6.3.3    | 固定候选方案的完成时间下界           | 63        |
| 6.3.4    | 在线择优与回退                 | 64        |
| 6.3.5    | 时间复杂度分析                 | 66        |
| 6.4      | 结果与分析                   | 67        |
| 6.4.1    | 共享缓存的配对实验               | 67        |
| 6.4.2    | 启发式选择条件的消融分析            | 68        |
| <b>7</b> | <b>综合实验与结果分析</b>        | <b>71</b> |
| 7.1      | 实验设置                    | 71        |
| 7.2      | 求解质量与计算效率               | 72        |
| 7.2.1    | 多核加速比比较                 | 72        |
| 7.2.2    | 求解耗时与评测开销               | 74        |
| 7.3      | 方法比较与局限性                | 75        |
| 7.3.1    | 算法版本比较与消融               | 75        |
| 7.3.2    | 完成时间下界与最优性差距            | 76        |
| 7.3.3    | 实验结论的适用范围               | 77        |

|                                |    |
|--------------------------------|----|
| <b>8 模型评价与总结</b>               | 78 |
| 8.1 方法特点                       | 78 |
| 8.2 主要结果                       | 79 |
| 8.3 模型局限性                      | 79 |
| 8.4 结论                         | 79 |
| <b>参考文献</b>                    | 80 |
| <b>附录 A 全量用例测试结果与数据说明</b>      | 82 |
| <b>附录 B 补充数学定理与推导证明</b>        | 82 |
| B.1 基于数据依赖的完成时间下界证明            | 82 |
| B.2 时间区间工作量与不可分割性下界            | 82 |
| <b>附录 C 程序与数据来源</b>            | 83 |
| C.1 算法实现版本与源码附件                | 83 |
| C.2 问题二全局理论下界证书                | 84 |
| <b>附录 D 算例 case_051 的执行时间线</b> | 84 |

# 1 问题重述与分析

## 1.1 问题背景与优化目标

本题面向神经网络处理器（neural processing unit, NPU）的多核调度优化问题。NPU 包含多个计算核心（NPU core，下文统称为“核心”，图表中简称为 Core）。NPU 代表整颗处理器芯片系统，而核心是指其内部并行的独立计算与控制单元。每个核心配备专用的矩阵计算单元 Cube、向量计算单元 Vector、核内核外数据搬运单元，以及该核心独占访问的私有片上缓存——局部存储器 L1 与统一缓冲区（Unified Buffer，简称 UB）。各核心之间无法直接相互读写对方的私有存储空间，所有核心通过总线共享核外主存储器，题面简称为 DDR（源自双倍数据率动态随机存取内存，double data rate dynamic random-access memory）。DDR 的总读写带宽、片上私有缓存容量以及各类硬件操作之间的同步等待周期均由硬件系统固定。

交由 NPU 执行的计算任务，在底层由一系列具有前后依赖关系的算子运算及其读写的数据组成。一个算子通常需要等待其前驱算子输出所需的数据后方可启动。为了形式化地刻画这些运算任务及其时序与数据流动约束，题面将待执行的神经网络模型表示为**计算图（computational graph）**。算法的目标是在给定的硬件约束下，共同决定计算任务的操作划分、核心分配以及执行次序 [1]。

以两次相互独立的矩阵乘计算为例，将它们分别分配至两个核心可以实现时间上的并发执行；然而，若两项计算均需从核外 DDR 载入大规模输入张量，它们就会在总带宽有限的 DDR 访问通道上产生争用。此外，若后续运算必须等待这两次矩阵乘全部结束方能汇总，系统还必须计入数据传输延迟与跨核心同步等待。由此可见，单纯平衡各核心的计算量只是调度优化的基础前提，完成时间的长短更取决于数据搬运开销、共享带宽争用以及依赖等待链的综合作用。

本题的首要评价指标为总体任务执行时间（Makespan），即在官方离散事件模拟评估程序下，从所有操作的最早启动时刻到全部操作均完成的最晚结束时刻所经历的硬件时钟周期数（cycle）。与方案质量指标相对应的是算法本身的求解效率，即算法程序在通用计算设备上从读取输入图到持久化输出合法方案所耗费的墙钟时间（Wall-clock Time），本文统称为求解时间（以秒为单位）。前者衡量调度方案在模拟环境下的执行性能，后者反映算法的实用工程效率。本文报告的执行时刻与周期数均源自官方评测系统的模拟推进，不作为物理硬件测量的绝对性能指标。

## 1.2 计算图与硬件执行约束

### 1.2.1 计算图表示与数据依赖

题面将神经网络抽象为二部有向无环图（bipartite directed acyclic graph, bipartite DAG）形式的计算图。图中的节点集合被严格划分为两类互不相交的顶点：算子节点（Op node，表示一次具体的数学计算或数据搬运）与数据张量节点（Tensor node，表示输入数据、计算中间结果或最终输出数据）。

有向边仅连接不同类别的节点，表达数据的生成与消费逻辑：若算子  $u$  运算生成数据张量  $t$ ，则计算图中存在有向边  $u \rightarrow t$ ；若算子  $v$  需要读取张量  $t$  作为计算输入，则存在有向边  $t \rightarrow v$ 。图中不存在算子到算子或张量到张量的直接连边。张量节点包含 **size** 属性，记录该数据块所占用的物理存储大小（以字节 Byte 为单位）。

根据计算图的有向拓扑，操作之间存在两类依赖关系：若算子  $u$  生成张量  $t$ ，且算子  $v$  直接读取该张量  $t$ （即二部图中仅通过单一中间张量节点连接  $u \rightarrow t \rightarrow v$ ，且不经任何

其他计算操作)，则称操作  $v$  **直接数据依赖** 于操作  $u$ ；此时，后继操作  $v$  的开始时刻不得早于前驱操作  $u$  的完全完成时刻。若从操作  $u$  到操作  $v$  的有向路径中途经过了一个或多个其他中间操作（即形成了  $u \rightarrow \dots \rightarrow w \rightarrow \dots \rightarrow v$  的传递链条），则称操作  $v$  **间接数据依赖** 于操作  $u$ 。只要两个操作之间存在直接或间接数据依赖，后继操作  $v$  的开始时刻就必须滞后于前驱操作  $u$  的结束时刻。若两个计算操作仅共同读取同一份外部输入张量，仅就该输入共享关系本身而言并不构成先后依赖；但二者仍可能在计算图中通过其他路径存在先后数据依赖，需结合全图拓扑综合判定。计算图的有向无环特性确保了计算流单向推进，不存在自相矛盾的时序闭环。

在后续算法设计与拓扑分析中，为了清晰展示计算任务的先后偏序关系，我们常将二部图向仅包含算子节点的**算子依赖图**进行投影：对二部图中每一个张量节点  $t$ ，若算子  $u$  是生成  $t$  的操作（ $u \rightarrow t$ ），且操作集合  $\{v \mid t \rightarrow v\}$  读取  $t$ ，则在算子依赖图中由  $u$  向该集合中的每一个读取算子  $v$  引出一条有向依赖边  $u \rightarrow v$ 。算子依赖图是用于简化依赖推导的辅助表示，底层的存储占用、搬运开销以及缓存判定仍基于带有张量大小属性的原始输入图展开。

根据官方输入输出契约，题面预设了两类核外数据搬运操作：**COPY\_IN** 负责将指定张量从核外 DDR 搬运至对应核心的私有缓存（L1 或 UB）；**COPY\_OUT** 负责将计算核心内部私有缓存中的数据写回至 DDR。算子的唯一标识编号（Op ID）与张量的标识编号（Tensor ID）属于两个独立的命名空间，分别唯一确定底层的调度单元与数据对象。

### 1.2.2 执行资源与计算通信重叠

在每个核心内部，物理硬件被组织为若干专用的硬件执行管道（Pipe），题面将其运行机制统称为**硬件流水线（pipeline）**。在本文所针对的 NPU 体系结构语境下，硬件流水线（pipeline）特指核心内部服务于特定指令类型的微架构执行资源及其严格保序的处理机制。根据题面硬件建模，每个核心包含四条相互平行的 Pipe：

1. **PIPE\_M**：驱动矩阵计算单元 Cube，专门负责矩阵乘累加等密集矩阵算子；
2. **PIPE\_V**：驱动向量计算单元 Vector，专门负责向量算术、非线性激活等向量算子；
3. **PIPE\_MTE2**：驱动核内数据搬运单元，负责将数据从核外 DDR 搬入私有缓存 L1/UB，或在私有缓存内将数据从 UB 转移至 L1；
4. **PIPE\_MTE3**：驱动核内数据搬运单元，负责将私有缓存 L1/UB 中的数据写回至 DDR，或在私有缓存内从 L1 转移至 UB。

官方离散事件评测程序对流水线推进作出了明确的时序约束：在单个核心内部，同一条 Pipe 在任意时刻至多只能执行一个操作；不同 Pipe 之间只要满足数据依赖和片上存储容量条件，则允许在时间轴上并发推进。

例如，当核心内的向量流水线（PIPE\_V）正在对已位于私有缓存中的数据执行运算时，搬运流水线（PIPE\_MTE2）可以同时从核外 DDR 载入后续算子所需的其他输入张量。两项操作如果在时间轴上拥有一段共同的执行区间，便在逻辑上达成了**计算与数据搬运的时间重叠（overlap of computation and data transfer）**。在模拟模型中，实现时间重叠能够有效减少处理器的空转等待。能否顺利实现时间重叠，取决于输入数据是否已传输到位、对应执行 Pipe 是否处于空闲状态、所属任务是否获准激活，以及私有缓存 L1/UB 中是否有足够的可用存储容量以容纳输出张量。若因等待前驱结果或缓存不足导致 Pipe 空闲停顿，系统时间线上就会产生调度气泡（Bubble）。

### 1.2.3 子图划分与调度方案

面向多核心体系架构，算法需要协同确定三项核心决策：

1. **操作划分**：将计算图中全部非核外搬运的待调度算子划分为若干互不重叠的集合，每个集合连同其引用的局部张量及依赖边构成原计算图的一个子图（**subgraph**）；
2. **核心分配**：将划分出的各个子图分别映射指派至特定的 NPU 核心负责处理；
3. **时序编排**：在同一个核心内部，确定该核心所承载的各个子图之间的执行先后次序。

根据官方提交契约（`contest_io.py`），提交方案必须且仅包含 `node_to_subgraph` 和 `core_schedules` 两个字段。其中 `node_to_subgraph` 映射字典负责记录原计算图中操作类型不属于 `COPY_IN` 与 `COPY_OUT` 的全部算子所属的子图编号（核内搬运或其他非核外 `COPY` 算子仍须纳入划分）；`core_schedules` 列表为每个核心分别给出该核心所分配子图编号的执行顺序序列。方案必须保证上述全部待调度算子被无遗漏、无重复地覆盖，且每个子图编号在所有核心的调度列表中恰好出现一次。

在评测系统的执行语义中，**执行任务（Task）** 是分配给特定核心执行的计算单元。子图是算法在计算图拓扑层面划分的操作集合，而 **Task** 则是评测模型运行时的调度与隔离边界，核心实际执行的是子图所对应的计算任务。赛题所提出的三个问题，其本质差异正是由子图到 **Task** 的组织关系以及片上数据驻留规则的改变所界定的：

**表 1-1 问题一至问题三的硬件执行机制与调度规则对比**

| 机制与规则维度    | 问题一                          | 问题二                                         | 问题三                             |
|------------|------------------------------|---------------------------------------------|---------------------------------|
| Task 的组织构成 | 每个子图独立构成一个 Task              | 同一核心所分配的全部子图合并为一个统一的 Task                   | 与问题二相同                          |
| 同核跨子图数据保留  | 严禁保留，必须经由 DDR 写回与重新载入        | 允许在私有缓存 L1/UB 中跨子图直接驻留与复用                   | 与问题二相同                          |
| 同一核心内切换延迟  | 连续 Task 间必须等待 100 cycles     | 核心仅执行单一合并 Task，无内部 Task 切换延迟                | 与问题二相同                          |
| 跨核心通信同步延迟  | 跨核前驱 Task 完成后需等待 1000 cycles | 对应源 <code>COPY_OUT</code> 完成后需等待 500 cycles | 与问题二相同，即使命中只读 Cache 也需满足该时序约束   |
| 共享二级缓存（L2） | 无，核心仅能访问私有 L1/UB 与共享 DDR     | 无，核心仅能访问私有 L1/UB 与共享 DDR                    | 引入核外共享只读缓存（Level-2 Cache，简称 L2） |

由上述规则演进可知：

- 问题一将每个子图视为隔离的 Task，算法面临增加多核并行度与增加 DDR 搬运及 Task 切换边界之间的直接冲突；
- 问题二通过在同一核心内将子图合并为大 Task，解除了同核跨子图的数据强制落盘限制，允许中间结果在私有片上缓存中直接驻留复用，但中间张量在缓存中停留越久，对私有存储空间的占用压力就越大；

- 问题三在多核合并执行的基础上引入了全局只读 Cache，多个核心读取相同张量时，若数据已驻留于 Cache 则产生**缓存命中 (cache hit)**，可直接从高速 Cache 通道读取而无需重复向 DDR 发起请求，从而有效分流共享带宽；若不在 Cache 中则发生**缓存未命中 (cache miss)**，需从 DDR 搬入并在搬入完成后按先进先出 (FIFO) 规则写入 Cache；单张量大小超过 Cache 总容量时则不予缓存。

图 1-1 针对相同的代表性计算片段，直观对比了三类问题在 Task 构成、数据流动路径以及多核心交互方式上的结构演变。

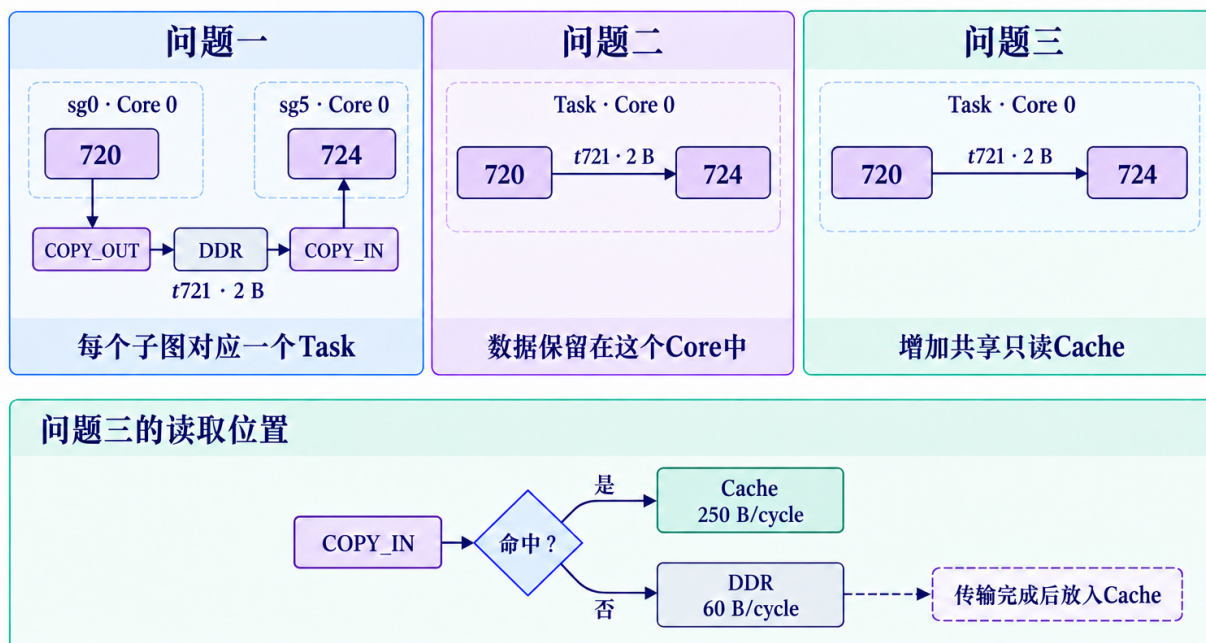


图 1-1 三问执行与存储机制对比

**图注：**三问条件的差别。问题一为图中不同子图所对应的操作分别建立 Task，数据经 DDR 中转；问题二允许同一个核心保留数据；问题三另为符合条件的 COPY\_IN 提供共享只读 Cache。下部示意数据能够装入 Cache 时的读取过程；所有请求仍须满足原依赖及规定等待。

### 1.3 问题分析与求解思路

综合题意与硬件执行特征，在通用神经网络处理器上实现高效的多核调度存在以下三项核心难点：

第一，**并行计算收益与跨核通信开销之间的非线性权衡**。将计算图拆解为更多子图并分散至更多核心，理论上能够提升计算单元的宏观并发度；然而，跨核心的数据交换必须插入额外的 COPY\_OUT 与 COPY\_IN 搬运操作。所有核心的 DDR 搬运操作均受制于总带宽仅为 60 字节/周期的共享内存通道，密集的跨核通信极易引发严重的内存带宽争用与同步停顿。如果因通信导致的额外 DDR 搬运与等待所增加的执行周期，超过了并行分配所节省的计算周期，多核方案的总完成时间反而会恶化。

第二，**片上私有缓存容量限制与数据驻留的动态约束**。在问题二和问题三中，将具有数据依赖的操作集中至同一核心能够避免外部内存搬运；然而，核心内部 L1 和 UB 的私有存储容量是严格受限的。核内调度在编译阶段若检测到局部缓存容量不足，会预先插入

将数据换出至核外 DDR、后续再换入的溢出搬运操作 (spill)。这不仅会增加额外的 DDR 搬运开销，若调度依赖安排不当还会导致方案无法推进。

第三，**局部核心排序与全局数据依赖交织下的死锁风险**。通过反例，我们可以说明为什么仅仅保证原始计算图无环并不能确保多核调度方案合法：假设核心 0 上的子图任务  $S_1$  需等待核心 1 上的子图任务  $S_2$  提供数据，而核心 1 内部指定的调度次序又要求先执行任务  $S_3$  再执行任务  $S_2$ ；若任务  $S_3$  恰好又间接依赖于任务  $S_1$  的运算结果，两个核心便会陷入“ $S_1$  等待  $S_2$ 、 $S_2$  等待  $S_3$ 、 $S_3$  等待  $S_1$ ”的循环依赖状态，导致评测系统发生死锁。

针对上述挑战，本文遵循题面关于避免暴力迭代搜索、单用例高效求解的指导原则，设计了结构化的求解流程：

1. **基于图结构的特征分析与候选构造**：分析计算图的连通分支、关键路径与张量复用关系，针对独立并行分支与数据密集型计算链，直接构造少量具有结构特征的完整候选方案；
2. **调度顺序的无环性判定**：将核心内子图执行次序作为时序边显式引入子图依赖图中，证明所给次序不产生有向环，确保方案满足依赖先后关系（需要说明的是，顺序无环是合法执行的必要条件，但仍需结合存储容量与完整评测进一步检验）；
3. **结合通信代价模型的理论下界快速剪枝**：基于题面给定的硬件流水线资源与通信带宽约束，计算理想并发条件下的时间下界与跨核通信开销，辅助快速筛除劣质候选；
4. **离散事件模拟选优**：在受限的评估预算内，调用评测程序对生成的完整候选方案进行离散事件模拟，从中选取 Makespan 最小的方案作为输出。

## 2 模型假设与符号说明

### 2.1 模型假设与执行约束

本文的建模与算法设计严格建立在题面给定的体系结构参数与官方离散事件评测程序的基础上。计算图中各算子的计算语义、操作类型及原始数据依赖保持不变，算法不得通过删减计算或修改运算结果来缩短完成时间。全部存储容量、总线带宽及同步等待常数均采用官方固定配置 [1]。

根据硬件配置参数，系统的主要物理约束如下：

1. **核心数量**：可用核心数  $K \in \{1, 2, 3, 4, 5\}$ ，系统为各核心编号为  $0, 1, \dots, K-1$ ；
2. **片上私有缓存**：每个核心配备独立的局部存储器 L1 与统一缓冲区 UB，其存储容量分别为 524,288 B (512 KiB) 与 131,072 B (128 KiB)。两级缓存各自独立管理，未分配的 L1 容量无法与 UB 共享；
3. **共享主存与缓存带宽**：所有核心访问核外主存 (DDR) 的读写操作共享统一的全局带宽池，总带宽固定为 60 B/cycle。在问题三中，引入容量为 1,048,576 B (1 MiB) 的共享只读二级缓存 (Level-2 Cache，简称 L2)，命中 Cache 的读取操作使用专用的共享 Cache 读取带宽通道，总带宽为 250 B/cycle，该带宽与 DDR 带宽相互独立，不占用 DDR 带宽池；
4. **同步与切换延迟**：在问题一中，同一核心连续执行的相邻 Task 之间须等待 100 cycles 的切换时间；若一个 Task 依赖位于其他核心的前驱 Task，其启动时刻还须满足晚于前驱 Task 完成时刻至少 1000 cycles 的跨核同步延迟。在问题二和问题三中，由于同核子图合并为单一 Task，核心内部不存在 Task 切换延迟；跨核心依赖仅作用于具体数据搬运操作，目标核心的 COPY\_IN 需在源核心对应的 COPY\_OUT 完成后等待 500 cycles 方可参

与就绪判断。这些常数由评测程序在对应的事件发生时检查，不直接等同于图上的固定边权重。

评测程序采用离散事件模拟推进执行。对于流水线操作队列，算子严格按核内静态调度生成的偏序队列以先进先出（first in, first out, FIFO）方式推进就绪与发射。对于问题三中的只读 Cache，其缓存行维护独立的 FIFO 淘汰队列：当 Cache 剩余容量不足以容纳新载入的张量时，系统依次淘汰最早写入 Cache 的条目；后续核心读取 Cache 发生命中时，只读访问不会更新该条目在 FIFO 队列中的淘汰顺序。单张量物理大小超过 Cache 总容量（1 MiB）的数据不进入 Cache。

若核内调度算法在静态编译时评估到私有片上缓存（L1 或 UB）容量不足以容纳计算过程中的所有活跃张量，评测程序会在该核心的指令流中预先插入溢出搬运操作（spill），即通过 PIPE\_MTE3 将暂不使用的中间数据经由 DDR 写出，后续需要时再通过 PIPE\_MTE2 重新从 DDR 读入。溢出搬运会产生额外的 DDR 访问流量并占用共享带宽，调度方案应尽量平抑片上张量驻留峰值以减少溢出开销。

## 2.2 张量与物理副本的表示

在计算图的生命周期中，同一份数学数据在不同调度阶段可能对应不同的图节点与存储实体，我们明确区分以下三类对象：

1. **原图张量节点**：指原始输入计算图 JSON 文件中给定的张量实体，具有唯一的整型 id 和固定的数据大小 size；
2. **调度生成的数据副本**：在多核心切分与核内调度阶段，当数据发生跨核心传输或因容量受限触发溢出时，评测程序会为该数据在不同存储介质（如源核心私有缓存、核外 DDR、目标核心私有缓存）中派生出新的物理数据实例。这些副本虽然承载相同的内容，但在物理上分别占用各自存储介质的独立空间；
3. **逻辑数据标识（logical\_tid）**：评测系统在处理问题三的只读 Cache 查询时，为所有源于同一张量派生出的物理副本赋予统一的逻辑标识 logical\_tid。当核心发起 COPY\_IN 搬运时，评测程序依据请求数据的 logical\_tid 查询 Cache 队列。若命中则直接从 Cache 读取，若未命中则在 DDR 读取完成后以该 logical\_tid 登记至 Cache 中。

此外，方案合法性必须满足三重判定条件：

- **操作覆盖合法**：输入图中除 COPY\_IN 与 COPY\_OUT 之外的全部算子，均被无重复、无遗漏地指派至唯一的子图，且每个子图在 core\_schedules 中恰好出现一次；
- **依赖拓扑无环**：子图间的数据依赖与各核心指定的子图执行顺序合并构成的扩展依赖图不存在有向环；
- **存储与执行可行**：评测程序对方案能够成功完成核内调度并推进离散事件模拟，不因容量冲突或死锁导致模拟异常终止。

只有评测程序完整运行成功并输出日志的方案，其导出的 Makespan 才是有效指标。若方案因违反依赖或容量超限被评测程序拒绝，或者执行发生超时，均视为无效解，不将其填为零参与均值统计。

## 2.3 符号说明

本文使用的主要数学符号定义如下表所示。

表 2-1 主要符号

| 符号                     | 定义说明                                                                       | 取值范围或量纲                                                                              |
|------------------------|----------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| $G = (O \cup T, E)$    | 原始计算图，其中 $O$ 为操作节点集合， $T$ 为张量节点集合， $E$ 为有向依赖边集合                            | 题面输入给定                                                                               |
| $V$                    | 待调度的计算算子集合（即 $O$ 中操作类型不为 <code>COPY_IN</code> 和 <code>COPY_OUT</code> 的算子） | $V \subset O$                                                                        |
| $b_t, \ell_t$          | 数据张量 $t$ 的物理大小（字节）及其预设存储位置（DDR、L1 或 UB）                                    | $b_t \in \mathbb{N}^+(\text{B})$ , $\ell_t \in \{\text{DDR}, \text{L1}, \text{UB}\}$ |
| $d_v, p_v$             | 算子 $v$ 的硬件执行周期数及其所绑定的硬件执行管道 Pipe                                           | $d_v \geq 1 (\text{cycle})$ , $p_v \in \{\text{PIPE\_M}, \text{PIPE\_V}, \dots\}$    |
| $K$                    | 系统可用的计算核心总数                                                                | $K \in \{1, 2, 3, 4, 5\}$                                                            |
| $\pi(v)$               | 算子划分映射，表示算子 $v$ 所归属的子图编号                                                   | $\pi : V \rightarrow \mathbb{N}$                                                     |
| $a(s)$                 | 子图分配映射，表示子图 $s$ 所指派执行的核心编号                                                 | $a : \mathcal{S} \rightarrow \{0, 1, \dots, K - 1\}$                                 |
| $\sigma_k$             | 核心 $k$ 所承担的子图编号的有序调度列表                                                     | 子图编号的全排列序列                                                                           |
| $P = (\pi, a, \sigma)$ | 完整的多核切分与调度方案（对应提交 JSON 文件的两个字段）                                            | 参赛提交规范                                                                               |
| $q$                    | 赛题问题编号                                                                     | $q \in \{1, 2, 3\}$                                                                  |
| $M_q(P)$               | 方案 $P$ 在问题 $q$ 的评测环境下运行所得的总体任务执行时间（Makespan）                               | cycle                                                                                |
| $D_q(P)$               | 方案 $P$ 在评测程序下产生的总额外 DDR 数据搬运量                                              | B（字节）                                                                                |
| $B_i$                  | 第 $i$ 张计算图在官方基准单核心调度算法下的基线 Makespan                                        | cycle                                                                                |
| $H_{i,K}$              | 第 $i$ 张计算图在 $K$ 个核心下运行问题三方案所得的 Cache 字节命中率                                 | $[0, 1]$                                                                             |